

Motion Planning in Uncertain Environments with Rapidly-Exploring Random Markov Decision Processes

Alex Rutherford, Paul Duckworth, Nick Hawes and Bruno Lacerda

Abstract— We propose rapidly-exploring random Markov decision processes (RRMDPs), a novel sampling-based motion planning approach for situations where the environment parameters are not fully known a priori, but a prior distribution over such parameters is available. Our algorithm combines ideas from established motion planning algorithms to achieve motion policies that are able to robustly drive the robot to its goal in the presence of uncertain action outcomes. We evaluate RRMDP in two domains, showing that it can synthesise motion policies that are more robust than the motion plans obtained by particle rapidly-exploring random trees (pRRT), a widely used algorithm for motion planning under uncertainty which RRMDP builds upon.

I. INTRODUCTION

The problem of motion planning is pivotal to the development of autonomous mobile robots. As such, many approaches have been developed to tackle it [1]. A particularly successful class of motion planning algorithms is the so called *sampling-based approaches* [2], e.g. probabilistic roadmaps (PRMs) or rapidly-exploring random trees (RRTs). Whilst these approaches are widely used, they assume deterministic environments.

In this paper, we focus on motion planning in situations where there is uncertainty over the value of environmental parameters that influence the robot's motion. Examples of such parameters are the coefficient of friction of a terrain, which will influence wheel slippage of an autonomous rover; or underwater currents which will push an autonomous underwater vehicle. A widely used extension of RRTs to deal with uncertain environments is the particle rapidly-exploring random trees (pRRT) algorithm [3]. pRRT deals with the uncertain environment by treating the extension process of the search tree as a stochastic process that is simulated multiple times, according to a distribution over the environmental parameters. While it has been shown that pRRT can find motion plans that reach the goal under uncertainty, it has several limitations. These are related to the low probability of reaching the goal in the search tree. Specifically, since each transition in the pRRT search tree is stochastic, the probability of successfully following a path in the tree quickly diminishes with path length. To mitigate this issue, pRRT biases the growth of the tree to only accept extensions from nodes that can be reached from the initial node with high enough probability. However, this leads to the algorithm rejecting a large percentage of possible extensions as the tree grows. Furthermore, pRRT heavily relies on the existence of a controller that is able to correct endpoint error, i.e., the distance between the endpoint of the planned path and the endpoint arrived at during execution. The controller is thus required to ensure that the produced

motion plan is followed by the robot. An evaluation of different controllers in this context can be found in [3].

We present an approach that does not require biasing and reduces the reliance on the controller. This is achieved by extending the search procedure such that it incrementally builds a Markov decision process (MDP) instead of a tree. To do so, we combine pRRT with the rapidly-exploring random graph (RRG) algorithm [2], an extension of RRT that builds a search graph instead of a search tree. This allows for the synthesis of *motion policies* that map each position visited by the robot to the optimal navigation action to be taken at that position. This improves robustness and reduces the reliance on the external controller by providing actions to be executed in states outside of the motion plan that would be obtained by pRRT.

The main contributions of this paper are (i) the formalisation of the motion planning under uncertainty problem as a cost-optimal reachability problem in an MDP; and (ii) the rapidly-exploring random MDP (RRMDP) algorithm, a sampling-based algorithm that combines pRRT and RRG to incrementally build an MDP that can be used to synthesise motion policies that are robust to environmental uncertainty. We evaluate RRMDP in two domains, showing that it outperforms pRRT in terms of robustness of the obtained motion plan.

II. RELATED WORK

Several works have extended RRT to deal with environmental uncertainty, representing uncertainty with alternative methodologies to the particle representation of pRRT. Relevant examples are [4], which uses the stochastic response surface method [5]; [6], which employs chance-constraints [7]; and [8], which uses linear-quadratic Gaussian motion planning [9]. The idea used in pRRT of sampling a trajectory multiple times to gain the likelihood of success also features in methods not based on an RRT framework. [10] introduces Monte Carlo motion planning, which uses a variance-reduced Monte Carlo algorithm to estimate the collision probability of a path through sampling many realisations. Note that all the approaches above still produce a tree and thus have similar limitations to the ones identified for pRRT.

There have also been many works focusing on incorporating *localisation uncertainty* to the motion planning problem. [11] uses a similar particle framework to pRRT to incorporate RRT within a simultaneous planning and mapping framework. Rapidly-exploring random belief trees [12] use an RRT* [2] and RRG framework to sample and refine trajectories to optimise the trade-off between information gathering (which is used to avoid obstacles) and path length. However, this algorithm assumes a perfectly known environment. Similarly, belief roadmaps (BRMs)

The authors are with the Dept. of Engineering Science, University of Oxford, UK. alexander.rutherford@pmb.ox.ac.uk, {pduckworth, nickh, bruno}@robots.ox.ac.uk.

Algorithm 1: Main rapidly-exploring algorithm loop.

```
1 Function BuildGraph( $x_{\text{init}}$ )
2    $V \leftarrow \{x_{\text{init}}\}; E \leftarrow \emptyset;$ 
3   for  $i = 1, \dots, n$  do
4      $x_{\text{rand}} \leftarrow \text{Sample}();$ 
5      $(V, E) \leftarrow \text{Extend}(V, E, x_{\text{rand}});$ 
6   return  $(V, E)$ 
```

Algorithm 2: RRT extension.

```
1 Function ExtendRRT( $V, E, x$ )
2    $x_{\text{nearest}} \leftarrow \text{Nearest}(V, x);$ 
3    $x_{\text{new}} \leftarrow \text{Steer}(x_{\text{nearest}}, x);$ 
4   if CollisionFree( $x_{\text{nearest}}, x_{\text{new}}$ ) then
5      $V \leftarrow V \cup \{x_{\text{new}}\};$ 
6      $E \leftarrow E \cup \{(x_{\text{nearest}}, x_{\text{new}})\};$ 
7   return  $(V, E)$ 
```

propagate beliefs over a PRM and use an extended Kalman filter covariance factorisation to improve computational efficiency [13]. BRMs have been extended to be more robust [14] and efficient search approaches over BRMs have been proposed [15]. These methods focus on localisation and state uncertainty, in contrast with our work, which focuses on environmental uncertainty instead.

III. PRELIMINARIES

A. Sampling-based motion planning

The motion planning problem is the problem of finding a sequence of positions in a workspace that drives a robot from its initial location to a goal location. Sampling-based methods are one of the most widely used approaches for motion planning, due to their ability to quickly find solutions. In this work, we focus on RRTs [16] and extensions thereof. Given that RRMDP directly builds from RRG and pRRT, in this section we describe these algorithms thoroughly, adapting the notation used in [2] and [3]. In particular, we use similar *primitive procedures* for sampling, steering, etc. The meaning of each procedure is intuitive and we will explain them in words. We direct the reader to the corresponding papers for the formal definitions.

Central to this class of algorithms is the incremental construction of a navigation graph $\mathcal{G} = (V, E)$, where V is a finite set of nodes corresponding to collision-free points in the workspace of the robot and $E \subseteq V \times V$ is a set of edges corresponding to obstacle-free connections between nodes. Algorithm 1 represents the main search loop, which is shared by RRT, RRG and pRRT. The algorithm incrementally builds a navigation graph, starting from the robot's initial position by sampling points in the workspace of the robot and extending the navigation graph given each sampled point. RRT, RRG and pRRT each correspond to a specific extension algorithm, as detailed below.

RRT. RRT [16] has the most basic extension function, which is described in Algorithm 2.

The RRT extension starts by finding the node $x_{\text{nearest}} \in V$ that is nearest to the sampled point (line 2). It then returns a point x_{new} that represents moving from x_{nearest} towards the sampled point for a prespecified maximum length (line 3). If the path between x_{nearest} and x_{new} is collision-free, then x_{nearest} and an edge between x_{nearest}

Algorithm 3: RRG extension.

```
1 Function ExtendRRG( $V, E, x$ )
2    $x_{\text{nearest}} \leftarrow \text{Nearest}(V, x);$ 
3    $x_{\text{new}} \leftarrow \text{Steer}(x_{\text{nearest}}, x);$ 
4   if CollisionFree( $x_{\text{nearest}}, x_{\text{new}}$ ) then
5      $V \leftarrow V \cup \{x_{\text{new}}\};$ 
6      $E \leftarrow E \cup \{(x_{\text{nearest}}, x_{\text{new}}), (x_{\text{new}}, x_{\text{nearest}})\};$ 
7     for  $x_{\text{near}} \in \text{Near}(V, x_{\text{new}}) \setminus \{x_{\text{nearest}}\}$  do
8       if CollisionFree( $x_{\text{near}}, x_{\text{new}}$ ) then
9          $E \leftarrow E \cup \{(x_{\text{near}}, x_{\text{new}}), (x_{\text{new}}, x_{\text{near}})\};$ 
10  return  $(V, E)$ 
```

and x_{new} are added to the navigation graph (lines 4–6). Note that the navigation graph this extension yields is actually a tree. Typically, RRT stops when a node within a prespecified distance to the goal is added to the navigation tree, with the output sequence to the goal being reconstructed by moving backwards in the tree from the goal node to the initial node.

RRG. Solutions obtained by RRT can be very suboptimal since the extension function adds new nodes to the tree greedily. The RRG algorithm mitigates this by building a graph structure instead. The RRG extension is described in Algorithm 3.

RRG [2] extends $\mathcal{G}$ similarly to RRT, with the main differences being that an edge from x_{new} to x_{nearest} is also added (line 6); and that edges are also added to all nodes within a set radius of x_{new} for which there is a collision-free path (lines 7–9). After n extensions, a shortest path algorithm, such as Dijkstra's algorithm [17], can be run in $\mathcal{G}$ to find the shortest path from the initial node to one of the nodes in V within the prespecified distance to the goal (if at least one such node exists). This allows for finding motion plans with lower cost than RRT, at the expense of more effort in the sampling phase, and the added need for a graph search algorithm.

pRRT. Both RRT and RRG assume that all relevant parameters of the environment are known, which allows for finding x_{new} using a deterministic *Steer* function. In many cases, this is too strong of an assumption. For example, a planetary rover might not be fully certain of the coefficient of friction of the terrain it must navigate on; or an underwater robot might have uncertainty over the direction and strength of currents it might face. In [3], Melchior & Simmons introduced pRRT, a particle-based RRT whereby uncertainty is added to the RRT extension, which is simulated a number of times. The extension function for pRRT is described in Algorithm 4.

The first difference from RRT is that pRRT assumes that there is a prior $\mathbf{F}$ over some environmental parameters (such as coefficient of friction or currents), and one can sample several instantiations of such parameters according to the probability density function of the prior (lines 5 and 6). For each instantiation, w of the world parameters, a particle x_{new} is calculated by moving from x_{nearest} towards the sampled point for a prespecified maximum length, but now under the parameters w (line 7). For example, w might represent the water current velocity at x_{nearest} , and the state x_{new} returned by *Steer* would take into account the

Algorithm 4: pRRT extension.

```

1 Function ExtendpRRT ( $V, E, x$ )
2    $x_{\text{nearest}} \leftarrow \text{Nearest}(V, x)$ ;
3   if  $x_{\text{nearest}}$  passes quality test then
4      $X_{\text{new}} \leftarrow \emptyset$ ;
5     for  $i = 1, \dots, k$  do
6        $w \leftarrow \text{SampleWorldParams}(\mathbf{F})$ ;
7        $x_{\text{new}} \leftarrow \text{Steer}(x_{\text{nearest}}, x, w)$ ;
8       if CollisionFree( $x_{\text{nearest}}, x_{\text{new}}$ ) then
9          $X_{\text{new}} \leftarrow X_{\text{new}} \cup \{x_{\text{new}}\}$ ;
10     $\{C_1, \dots, C_m\} \leftarrow \text{Cluster}(X_{\text{new}})$ ;
11    for  $i = 1, \dots, m$  do
12       $V \leftarrow V \cup \overline{C_i}$ ;  $E \cup \{(x_{\text{nearest}}, \overline{C_i})\}$ ;
13  return ( $V, E$ )

```

effect of the current on the robot's trajectory. All x_{new} that pass the collision test in line 8 are then clustered (line 10). Hierarchical clustering [18] based on the Euclidean distance between the particles is used. Then, for each cluster C_i , we add the mean of the particles in C_i , $\overline{C_i}$, to V , and the corresponding edge from x_{nearest} to $\overline{C_i}$ to E (line 12). As stated earlier, the most noticeable challenge with pRRT is the low probability to reach the goal. Since each transition in the tree is stochastic the probability of reaching nodes in the tree quickly diminishes with path length. To mitigate this, pRRT is biased toward generating paths that are more likely to be followed by the robot. This bias is achieved by only extending towards sampled nodes when their nearest node x_{nearest} passes a quality check (line 3). The quality of a node is a function of the probability of reaching that node in the tree and the minimum probability across all leaf nodes in the tree. This is formally defined in [3]. Note that these probabilities can be maintained as the tree is constructed, as we will show when presenting RRMDP. Finally, a node passes the quality check with a probability proportional to its quality value. Without this check, path probabilities are extremely low and thus it is not feasible to follow them practice. As such, in our experiments, we do not assess the performance of pRRT with this check relaxed.

B. MDPs and Cost-Optimal Reachability

An MDP is defined as $\mathcal{M} = (S, s_{\text{init}}, A, T, c)$, where S is a finite set of states; $s_{\text{init}} \in S$ is the initial state; A is a finite set of actions; $T : S \times A \times S \rightarrow [0, 1]$ is a probabilistic transition function; and $c : S \times A \times S \rightarrow \mathbb{R}_{\geq 0}$ is a cost function representing, for example, the expected time or the expected energy required to execute an action.

We define the set of actions enabled in s as $A_s = \{a \in A \mid \text{exists } s' \in S \text{ such that } T(s, a, s') > 0\}$. A *path* through an MDP is a sequence $\sigma = s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} \dots$ where $T(s_i, a_i, s_{i+1}) > 0$ for all $i \in \mathbb{N}$. We denote the set of all paths of $\mathcal{M}$ starting from state s as $\text{Path}_{\mathcal{M}, s}$. Without loss of generality, we assume all MDP states have at least one action enabled, i.e. the MDP does not have deadlocks and a run of the MDP always yields an infinite-length path in $\text{Path}_{\mathcal{M}, s_{\text{init}}}$. This can be achieved by adding a self-loop to each sink state, i.e. each state for which $A_s = \emptyset$.

The choice of action to take at each step of the execution of an MDP is made by a *policy*. In this paper, we consider

deterministic, stationary policies, defined as functions $\pi : S \rightarrow A$ that map each state $s \in S$ to the action to execute in s . We denote the set of all such policies as Π . Given an MDP $\mathcal{M}$ and a policy $\pi \in \Pi$, we can define a probability measure $Pr_{\mathcal{M}, s}^{\pi}$ over the set of paths $\text{Path}_{\mathcal{M}, s}$ [19]. Furthermore, for a measurable function $X : \text{Path}_{\mathcal{M}, s} \rightarrow \mathbb{R}$, we write $E_{\mathcal{M}, s}^{\pi}(X)$ for the expected value of X with respect to $Pr_{\mathcal{M}, s}^{\pi}$.

In this work, we consider *cost-optimal reachability* problems for which the probability of satisfaction might be less than one. These involve identifying a cost-optimal policy that reaches a set of goal states with the highest probability possible. More formally, let $G \subset S$ be a set of goal states. We define the set of paths that reach G as:

$$\text{reach}_G = \{(s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} \dots) \in \text{Path}_{\mathcal{M}, s_0} \mid \text{exists } i \in \mathbb{N} \text{ such that } s_i \in G\}. \quad (1)$$

We will consider policies that are cost-optimal, in the sense that they minimise the expected cumulative cost to reach either a goal state, or a state where reaching the goal is not possible. Given $\sigma = s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} \dots \in \text{Path}_{\mathcal{M}, s_0}$, we define l_{σ} as the timestep until which cost will be accumulated for path σ :

$$l_{\sigma} = \begin{cases} \min_l s. t. s_l \in G & \text{if } \sigma \in \text{reach}_G \\ \min_l s. t. \\ Pr_{\mathcal{M}, s_l}^{\max}(\text{reach}_G) = 0 & \text{otherwise,} \end{cases} \quad (2)$$

where $Pr_{\mathcal{M}, s_l}^{\max}(\text{reach}_G)$ denotes the supremum over Π of $Pr_{\mathcal{M}, s_l}^{\pi}(\text{reach}_G)$. Finally, consider the function $\text{cumul}_G : \text{Path}_{\mathcal{M}, s} \rightarrow \mathbb{R}_{\geq 0}$ that maps a path $\sigma = s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} \dots$ to the cost accumulated up to l_{σ} :

$$\text{cumul}_G(\sigma) = \sum_{i=0}^{l_{\sigma}-1} c(s_i, a_i, s_{i+1}). \quad (3)$$

Defining the set of policies that maximise the probability of reaching G as:

$$\Pi_G = \{\pi \in \Pi \mid \pi = \arg \max_{\pi'} Pr_{\mathcal{M}, \bar{s}}^{\pi'}(\text{reach}_G)\}, \quad (4)$$

the optimisation objective for constrained reachability can now be defined as finding the policy $\pi_G \in \Pi_G$ that has minimal expected cumulative cost:

$$\pi_G = \arg \min_{\pi \in \Pi_G} E_{\mathcal{M}, \bar{s}}^{\pi}(\text{cumul}_G). \quad (5)$$

The above optimisation problem is a variant of a safest and stochastic shortest path problem [20], with the minimal expected cost being known to converge to a finite value. To find π_G , we use the *nested value iteration* algorithm presented in [21]. This approach relies on extending value iteration to consider lexicographic multi-objective optimisation, with probability maximisation being the primary objective and cost minimisation being the secondary objective.

IV. RRMDP

A significant drawback of pRRT is that the quality check rejects the majority of possible extensions once the tree gets to a certain size. This has a detrimental effect on the algorithm's ability to find the goal. This insight motivates RRMDP, which overcomes these difficulties by maintaining

Algorithm 5: Main RRMDP algorithm loop.

```

1 Function BuildMDP ( $x_{\text{init}}$ )
2    $S \leftarrow \{x_{\text{init}}, \perp\}$ ;  $s_{\text{init}} \leftarrow x_{\text{init}}$ ;  $A \leftarrow \emptyset$ ;  $T \leftarrow$ 
   null;  $c \leftarrow$  null;
3   for  $i = 1, \dots, N$  do
4      $x_{\text{rand}} \leftarrow \text{Sample}()$ ;
5      $(S, A, T, c) \leftarrow \text{Extend}_{\text{MDP}}(S, A, T, c, x_{\text{rand}})$ ;
6   return  $(S, s_{\text{init}}, A, T, c)$ 

```

Algorithm 6: RRMDP extension.

```

1 Function ExtendMDP ( $S, A, T, c, x_{\text{rand}}$ )
2    $x_{\text{nearest}} \leftarrow \text{Nearest}(S, x_{\text{rand}})$ ;
3   for  $x \in S$  do
4      $X_{\text{new}}^x \leftarrow \emptyset$ ;
5     for  $i = 1, \dots, k$  do
6        $w \leftarrow \text{SampleWorldParams}(\mathbf{F})$ ;
7        $x_{\text{new}} \leftarrow \text{Steer}(x_{\text{nearest}}, x_{\text{rand}}, w)$ ;
8       if CollisionFree( $x_{\text{nearest}}, x_{\text{new}}$ ) then
9          $X_{\text{new}}^x \leftarrow X_{\text{new}}^x \cup \{x_{\text{new}}\}$ ;
10    if  $X_{\text{new}}^x \neq \emptyset$  then
11      for  $x_{\text{near}} \in \text{Near}(S, \overline{X_{\text{new}}^x}) \setminus \{x_{\text{nearest}}\}$  do
12        for  $i = 1, \dots, k$  do
13           $w \leftarrow \text{SampleWorldParams}(\mathbf{F})$ ;
14           $x_{\text{new}} \leftarrow \text{Steer}(x_{\text{near}}, \overline{X_{\text{new}}^x}, w)$ ;
15          if CollisionFree( $x_{\text{near}}, x_{\text{new}}$ ) then
16             $X_{\text{new}}^x \leftarrow X_{\text{new}}^x \cup \{x_{\text{new}}\}$ ;
17     $X_{\text{new}} \leftarrow \bigcup_{x \in S} X_{\text{new}}^x$ ;
18     $\{C_1, \dots, C_m\} \leftarrow \text{Cluster}(X_{\text{new}})$ ;
19     $S \leftarrow S \cup \{C_1, \dots, C_m\}$ ;
20    for  $x \in S \setminus \{C_1, \dots, C_m\}$  do
21      if  $X_{\text{new}}^x \neq \emptyset$  then
22         $p_{\text{fail}} \leftarrow 1$ ;  $a \leftarrow \{(x, x_{\text{rand}})\}$ ;  $A \leftarrow$ 
         $A \cup \{a\}$ ;
23        for  $i = 1, \dots, m$  do
24           $p \leftarrow \frac{|X_{\text{new}}^x \cap C_i|}{k}$ ;
25           $T(x, a, C_i) \leftarrow p$ ;
26           $p_{\text{fail}} \leftarrow p_{\text{fail}} - p$ ;
27           $c(x, a, C_i) \leftarrow \text{dist}(x, C_i)$ ;
28           $T(x, a, \perp) \leftarrow p_{\text{fail}}$ ;
29  return  $(S, A, T, c)$ 

```

a graph structure. This increases the probability of reaching a node since multiple routes are maintained. To do so, it combines the graph structure of RRG with the particle extension method of pRRT. This builds a navigation MDP that can be solved via dynamic programming.

The main loop for RRMDP is described in Algorithm 5. It is analogous to Algorithm 1, but incrementally builds an MDP instead of a graph. Also note that the set of states is initialised as $S = \{x_{\text{init}}, \perp\}$ (line 2), where $\perp$ is a sink state that will be used to represent collision states. These must be explicitly represented in the MDP in order to have a complete transition function.

The $\text{Extend}_{\text{MDP}}$ function incrementally adds new states and transitions to the MDP, combining ideas from RRG and pRRT, and is described in Algorithm 6.

As with the extension functions presented before, it starts by calculating x_{nearest} (line 2). It then initialises a particle set per state (lines 3 and 4). For each state $x \in S$, X_{new}^x

will hold particles sampled from x towards x_{rand} .

As in pRRT, RRMDP then enters a loop where it samples k extensions of x_{nearest} towards x_{rand} (lines 5–9). If the set X_{new}^x of particles successfully extended (i.e. for which the extension was collision-free) from x_{nearest} towards x_{rand} is not empty, then RRMDP enters an RRG-like loop where it tries extensions from other nodes near the mean of the particles in X_{new}^x (lines 10–16). Note that this loop attempts pRRT-style connections (lines 12–16).

After attempting all the extensions, RRMDP does hierarchical clustering over the set of all sampled particles (lines 17–18). As in pRRT, the means of the particles of each of these clusters are added to the state space (line 19). We then enter a loop that defines the transition and cost functions for nodes from which extensions were attempted to the newly added cluster representatives (lines 20–28). For each state x for which extensions were successful, i.e. for which $X_{\text{new}}^x \neq \emptyset$ (lines 20 and 21), we create an action that represents moving from x towards x_{rand} (line 22). Then, for each cluster representative $\overline{C_i}$, the probability of moving from x to $\overline{C_i}$ is defined as the number of particles in X_{new}^x that are also in C_i over the total number, k , of particles sampled from x (lines 24 and 25). The cost function is defined as the distance between x and $\overline{C_i}$ (line 27). Finally, the remaining probability mass (corresponding to the attempts to extend x that were not collision-free) is assigned to the sink state (line 28).

Once the MDP is constructed, we define the set G as the set of states in S that are within the prespecified distance to the goal (if at least one such state exists). Then, nested value iteration can be run to find the policy π_G , as defined in Equation 5.

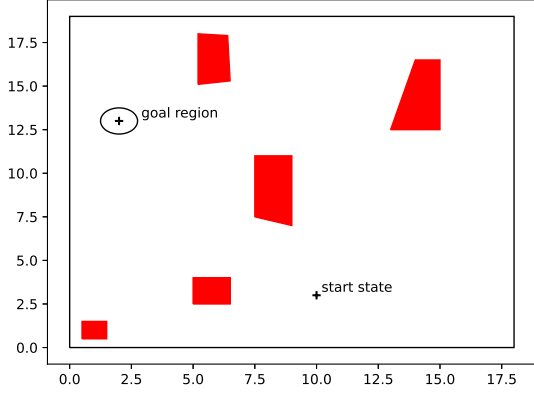
V. EXPERIMENTS

In this section, we provide an empirical study that shows that RRMDP is able to produce motion policies that outperform the plans produced by pRRT in terms of the probability of reaching the goal. We compared the algorithms over two domains. First, similar to the domain used in [3], we consider motion planning through a domain with varying elevation and an uncertain coefficient of friction. Then, we consider a domain obtained from real-world data, where an autonomous underwater vehicle (AUV) must navigate under uncertain currents.

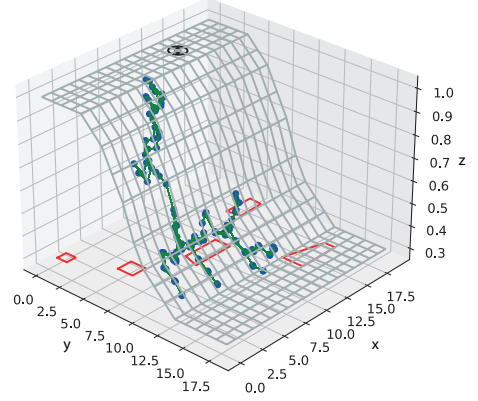
A. Rover domain

We start by comparing against pRRT in a domain similar to the one used in [3]. In this domain, an autonomous rover must plan motions on uneven terrain, subject to an unknown and varying coefficient of friction. In Fig. 1, we show a depiction of the problem instance the algorithms were tested on. The start state, goal region and obstacles (which are coloured red and extend for the whole vertical axis) are shown in Fig. 1a while the heightmap is presented in Fig. 1b, along with a partially built RRMDP graph. Fig. 1b depicts a heightmap with the steepest gradient of 15% (note the axis scales). We used the following discrete distribution over the coefficients of friction:

$$\begin{aligned} \text{c.o.f.} &= [0.1, 0.15, 0.2, 0.45, 0.5, 0.55], \\ \text{likelihood} &= [0.1, 0.25, 0.05, 0.15, 0.25, 0.2]. \end{aligned}$$



(a) Top view of workspace, start state, and goal region.



(b) Visualisation of the heightmap, along with a partially built RRMDP.

Fig. 1: Rover navigation domain.

TABLE I: Performance of pRRT and RRMDP over the friction domain (variance is bracketed).

Metric	pRRT	RRMDP
Goal found	6.67%	66.67%
Goal probability	$\sim 0\%$	20% (12%)
Mean runtime (s)	9.60 (0.94)	106.47 (58.49)
Mean model building time (s)	9.60 (0.94)	106.39 (58.25)
Mean model solving time (s)	0.03 (0.00)	0.13 (0.01)

Both algorithms were run 30 times, each for 300 iterations and with the same sampling sequence for each pair of runs. At each extension, 20 particles were sampled, with the c.o.f. for each was drawn according to its likelihood. Results for these tests are presented in Table I where “goal found” is the percentage of runs for which a node in the goal region was added to the search structure; and “goal probability” is the probability of reaching the goal whilst following the output path (pRRT) or the output policy (RRMDP). Note that the goal probability values are only for runs where the goal was found during model building.

While our method is more computationally expensive, the performance gains are notable. In particular, the absence of a quality check for expansion results in our method finding the goal at a much greater rate. Furthermore, the probability of following the sequence obtained by pRRT’s policy was essentially 0 (since as a path’s length increases, the probability of successfully following it tends to zero, as discussed earlier), while the RRMDP policy yields a mean probability of reaching the goal of 20%. While this does not completely remove the need for a controller to correct endpoint error, it reduces the reliance on a perfect controller, as the RRMDP policy can react to unwanted action outcomes. This is a consequence of the additional routes to the goal given by the graph structure, and the removal of the need for a quality check.

B. AUV domain

The second domain is adapted from [22] and uses a pre-trained Gaussian process (GP) model based on data from the Copernicus Marine Environment Monitoring Service [23] to

TABLE II: Performance of pRRT and pRRG over the currents domain (variance is bracketed).

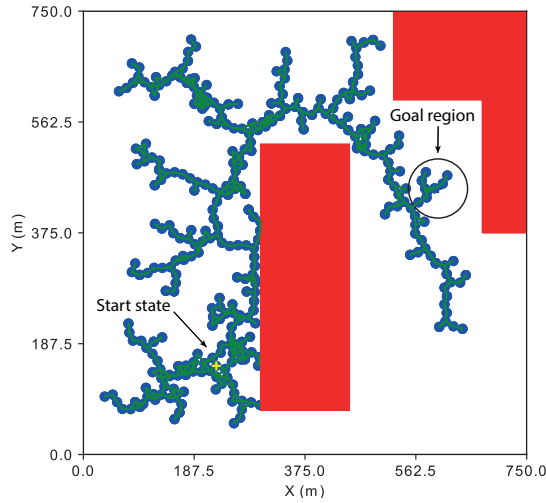
Metric	pRRT	RRMDP
Goal found	42%	78%
Goal probability	$\sim 0\%$	98% (1.8%)
Mean runtime (s)	5.97 (1.53)	20.87 (1.53)
Mean search time (s)	5.69 (0.91)	20.65 (1.99)
Mean path planning time (s)	0.68 (0.19)	0.27 (0.43)
Mean number of states	562 (18700)	830 (3150)

predict the currents an AUV will experience as it moves underwater. We use data from the North-West European Shelf (the North Sea). In this case, SampleWorldParams uses the GP prediction at each location to obtain a sample of the currents. The effect of the currents is modelled as a velocity applied to the AUV while it moves. We also added obstacles to represent areas that the AUV should avoid, e.g. due to safety concerns. The workspace is represented in Fig. 2a and the mean current velocities at our chosen time (hour 9 in the data) can be seen in Fig. 2b.

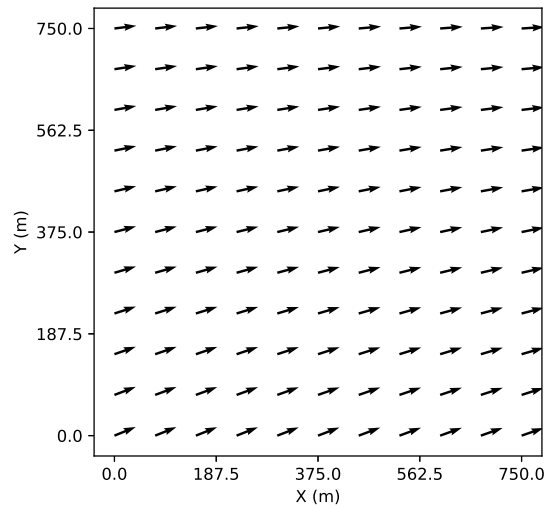
To improve the sampling process within RRMDP, the *Steer* function in line 12 of Algorithm 6 was amended to take into account the value of $X_{new}^{x_{nearest}}$. This value is used to gain an estimate of the mean current velocity and adjust the sampling direction of additional connections accordingly. This allowed for the construction of MDPs with more connectivity, which helped RRMDP achieve better results than in the previous domain, as shown next.

We ran pRRT and RRMDP 50 times, each for 400 iterations and with the same sampling sequence for each pair of tests. Once again, 20 particles were sampled at each extension. The quality check of pRRT was adjusted to increase the likelihood a node would be accepted, otherwise the success rate of the algorithm was very low.

The performance of the two algorithms, reported in Table II, was qualitatively similar to the previous domain. However, in this domain, RRMDP was able to build MDPs for which it was possible to ensure the AUV would reach its goal with probability close to 1. Thus, for this domain, the RRMDP policy can be executed without relying on



(a) View of workspace, start state, and goal region, along with a partially built RRMDP.



(b) Mean current velocities over workspace.

Fig. 2: AUV navigation domain.

a controller to correct endpoint errors. This allows for lightweight policy execution, as executing the RRMDP policy boils down to state estimation to update the policy state, and a table lookup to find the policy action. This makes the use of RRMDPs in these domains promising, as AUVs typically have low computed capabilities due to battery and weight limitations.

VI. CONCLUSION AND FUTURE WORK

In this paper, we presented RRMDP, a novel “rapidly-exploring” motion planning algorithm for uncertain environments. RRMDP addresses limitations of previous approaches by combining the graph structure of RRG with the particle filtering-based handling of uncertainty of pRRT. This enables the synthesis of motion policies that allow the robot to more robustly reach the goal, as highlighted in experiments in two different domains.

Future work includes dealing with temporal logic specifications; integrating sampling-based heuristic search approaches for MDPs [24] in the RRMDP construction, in order to achieve simultaneous model building and solving, thus improving the method’s efficiency; and experimenting on real uncertain environments.

ACKNOWLEDGMENT

This work was supported by UK Research and Innovation and EPSRC through the Robotics and Artificial Intelligence for Nuclear (RAIN) and the Offshore Robotics for Certification of Assets (ORCA) hubs [EP/R026084/1, EP/R026173/1], the EPSRC Programme Grant ‘From Sensing to Collaboration’ [EP/V000748/1], a gift from Amazon Web Services, and a Pembroke College Rokos Award.

REFERENCES

- [1] J.-C. Latombe, *Robot motion planning*. Springer Science & Business Media, 2012.
- [2] S. Karaman and E. Frazzoli, “Sampling-based algorithms for optimal motion planning,” *IJRR*, vol. 30, no. 7, 2011.
- [3] N. A. Melchior and R. Simmons, “Particle RRT for path planning with uncertainty,” in *ICRA*, 2007.
- [4] G. Kewlani, G. Ishigami, and K. Iagnemma, “Stochastic mobility-based path planning in uncertain environments,” in *IROS*, 2009.
- [5] S. Isukapalli, A. Roy, and P. Georgopoulos, “Stochastic response surface methods (SRSMs) for uncertainty propagation: application to environmental and biological systems,” *Risk analysis*, vol. 18, no. 3, 1998.
- [6] B. Luders, M. Kothari, and J. How, “Chance constrained RRT for probabilistic robustness to environmental uncertainty,” in *GNC*, 2010.
- [7] L. Blackmore, H. Li, and B. Williams, “A probabilistic approach to optimal robust path planning with obstacles,” in *ACC*, 2006.
- [8] L. Jaillet, J. Hoffman, J. Van den Berg, P. Abbeel, J. M. Porta, and K. Goldberg, “EG-RRT: Environment-guided random trees for kinodynamic motion planning with uncertainty and obstacles,” in *IROS*, 2011.
- [9] J. Van Den Berg, P. Abbeel, and K. Goldberg, “LQG-MP: Optimized path planning for robots with motion uncertainty and imperfect state information,” *IJRR*, vol. 30, no. 7, 2011.
- [10] L. Janson, E. Schmerling, and M. Pavone, “Monte carlo motion planning for robot trajectory optimization under uncertainty,” in *Robotics Research*. Springer, 2018.
- [11] Y. Huang and K. Gupta, “RRT-SLAM for motion planning with motion and map uncertainty for robot exploration,” in *IROS*. IEEE, 2008.
- [12] A. Bry and N. Roy, “Rapidly-exploring random belief trees for motion planning under uncertainty,” in *ICRA*, 2011.
- [13] S. Prentice and N. Roy, “The belief roadmap: Efficient planning in belief space by factoring the covariance,” *IJRR*, vol. 28, no. 11-12, 2009.
- [14] S. D. Bopardikar, B. Englot, A. Speranzon, and J. van den Berg, “Robust belief space planning under intermittent sensing via a maximum eigenvalue-based bound,” *IJRR*, vol. 35, no. 13, 2016.
- [15] T. Shan and B. Englot, “Belief roadmap search: Advances in optimal and efficient planning under uncertainty,” in *IROS*, 2017.
- [16] S. M. LaValle and J. J. Kuffner Jr, “Randomized kinodynamic planning,” *IJRR*, vol. 20, no. 5, 2001.
- [17] E. W. Dijkstra *et al.*, “A note on two problems in connexion with graphs,” *Numerische mathematik*, vol. 1, no. 1, 1959.
- [18] A. K. Jain, M. N. Murty, and P. J. Flynn, “Data clustering: a review,” *CSUR*, vol. 31, no. 3, 1999.
- [19] J. Kemeny, J. Snell, and A. Knapp, *Denumerable Markov chains*, 2nd ed. Springer-Verlag, 1976.
- [20] F. Teichteil-Königsbuch, “Stochastic safest and shortest path problems,” in *AAAI*, 2012.
- [21] B. Lacerda, F. Faruq, D. Parker, and N. Hawes, “Probabilistic planning with formal performance guarantees for mobile service robots,” *IJRR*, vol. 38, no. 9, 2019.
- [22] P. Duckworth, B. Lacerda, and N. Hawes, “Time-bounded mission planning in time-varying domains with semi-MDPs and Gaussian processes,” in *CoRL*, 2021.
- [23] M. Tonani, P. Sykes, R. R. King, N. McConnell, A.-C. Péquignot, E. O’Dea, J. A. Graham, J. Polton, and J. Siddorn, “The impact of a new high-resolution ocean model on the met office north-west european shelf forecasting system,” *Ocean Science*, vol. 15, 2019.
- [24] B. Bonet and H. Geffner, “Labeled RTDP: Improving the convergence of real-time dynamic programming,” in *ICAPS*, 2003.